第二次实验指导

模态调度与异构封装

同源复制 · 模态调度 · 异构封装

目录

第一部分：实验内容与任务

- 1.1 必要概念
- 1.2 实验内容
- 1.3 实验任务
- 1.4 配套材料

第二部分：实验过程及要求

- 2.1 Parser 功能完善
- 2.2 入网报文识别与关联信息生成
- 2.3 报文复制与模态调度
- 2.4 异构封装与路径转发
- 2.5 结果验证与原有功能检查
- 2.6 代码 TODO 与任务对应
- 2.7 过程记录
- 2.8 作业提交与完成检查

第三部分：实验指导

- 3.1 建立记录目录
- 3.2 运行端口映射模型
- 3.3 复制开发骨架并检查起点
- 3.4 完成 M1：协议头与 Deparser
- 3.5 完成 M2：Parser
- 3.6 完成 M3：Ingress 识别、序号、CRC 和复制
- 3.7 完成 M4：P4Runtime multicast 与 Egress 封装
- 3.8 启动修改后的项目
- 3.9 发送一份实验报文
- 3.10 检查 pcap 和交换机日志
- 3.11 检查原有功能
- 3.12 整理结果并保存版本
- 3.13 清理运行环境
- 3.14 参考运行现象

> 实验主题：基于 P4 的多模态网络内生安全
> 前置条件：已跑通原始三模态 Demo，并创建 `class1-baseline` tag

本次实验在原始 Demo 的 IPv4、IPv6 和 Source Routing 三条路径上继续开发：主机只发送一份实验报文，源网关在网络内部复制三份，再分别使用三种外层协议进入不同路径。本次先完成调度、复制、封装和路径验证；目的网关的多数裁决在下次实验继续完成。

第一部分：实验内容与任务

1.1 必要概念

1. 模态与异构转发

本实验把一种协议头、一套转发规则和一条网络路径组成的转发方式称为一个“模态”。IPv4 根据 IPv4 目的地址转发，IPv6 根据 IPv6 地址转发，Source Routing 则由报文携带路径信息。三种模态使用不同的协议头和路径，因此可以对同一份数据提供多个相互独立的转发结果。

2. 源网关、目的网关与方向

源网关是主机报文进入多模态网络的第一台网关交换机，负责识别实验流量、复制报文并进行异构封装。目的网关是多条路径的汇聚点，后续负责比较各模态结果并恢复原始报文。

网关角色会随通信方向互换：同一台网关在一个方向可能负责调度，在反向则可能负责汇聚结果。报文内需要保留方向信息，以便网关选择正确的处理分支。

3. 内层报文、外层报文与封装

主机原本发送的 IPv4 报文称为“内层报文”。源网关为它加上 IPv4、IPv6 或 Source Routing 头，得到用于网络内部转发的“外层报文”，这个过程称为封装。目的网关删除网络内部的外层信息，恢复主机原本报文的过程称为解封装。

主机可以使用 UDP 或 TCP 传输实验数据。两种方案的多模态处理思路相同，但 Parser、CRC 字段范围和收发脚本必须与实际选择的传输层协议一致。

4. PolyShim 与受保护数据

`PolyShim` 是放在外层模态头和内层报文之间的实验协议头。它保存报文方向、序号、模态编号、入口校验值和内层协议类型，使三种外层报文可以使用同一套后续处理逻辑。

`protected_data` 是为了便于 Parser 提取和完整性比较而定义的定长实验数据。固定长度使 Parser 能够按确定位宽提取字段，不代表 P4 可以随意读取任意长度的应用层负载。

5. 报文序号与 CRC32

`sequence_number` 是源网关为每份原始实验报文分配的序号。一份原始报文复制出的三份副本使用同一序号，目的网关才能知道它们属于同一组。

CRC32 把若干报文字段计算为一个 32 位校验值。`source_crc32` 是源网关在报文进入多模态路径前计算的参考值，后续可用来检查指定字段是否发生变化。CRC32 用于实验中的错误检测，不是密码学认证方法。

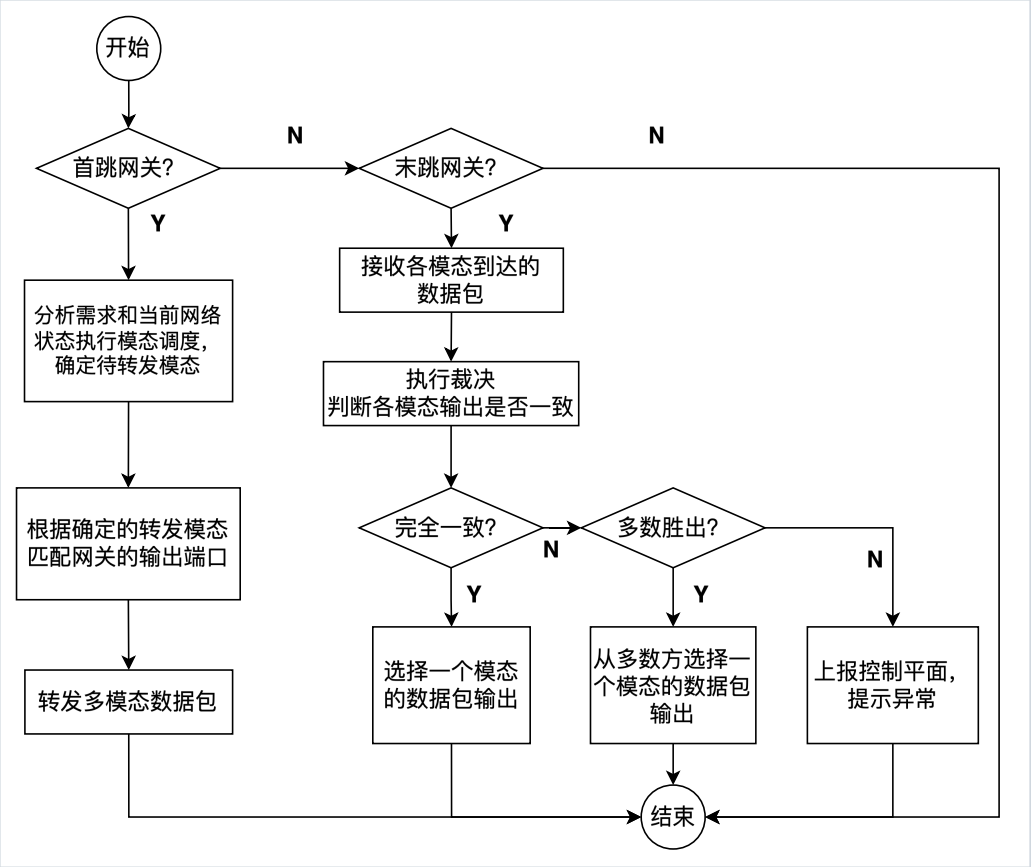
6. Parser、Ingress、PRE 与 Egress

- `Parser` 按协议格式从报文中提取 header，为后面的表和 action 提供字段。
- `Ingress` 是入口处理流程，负责识别报文、查表、准备关联信息并触发复制。
- `PRE`（Packet Replication Engine）是 v1model 中的报文复制模块，它根据控制器配置的 multicast group 产生多个副本。
- `Egress` 是每个副本离开交换机前的处理流程，可以根据已确定的出口选择外层封装。

- `Deparser` 把处理后仍然有效的 header 按线序重新写回报文。`metadata` 则是交换机处理报文时使用的内部临时信息，不会自动出现在线上报文中。

P4 数据平面负责逐包处理，P4Runtime 控制平面负责向交换机下发表项和 multicast group。P4 编译后生成的 `P4Info` 描述可供控制器访问的表、action、字段和参数。数据面与控制面使用的名称、字段和编号需要保持一致。

7. 整体处理流程



本次实验完成图中源网关一侧的报文识别、模态调度、出口匹配和多模态转发。图中目的网关的裁决和输出在下次实验完成。

1.2 实验内容

在原始三模态 Demo 上增加模态调度逻辑。实验报文从 h1 或 h2 发出后，源网关识别该报文，生成用于关联副本的信息，在网络内部复制三份，并分别形成 IPv4、IPv6 和 Source Routing 外层报文。通过收发记录、pcap 或交换机日志，说明一份输入已经进入三条对应路径。

1.3 实验任务

1. **Parser 功能完善**：识别本组选用的 UDP 或 TCP 实验报文，以及三种外层模态报文。
2. **入网报文识别与关联信息生成**：区分实验流量和普通流量，并为同一原始报文的副本生成共享序号和校验值。
3. **报文复制与模态调度**：在源网关产生三份相关副本，并将它们分配到三个模态出口。
4. **异构封装与路径转发**：为三份副本生成不同的外层协议头，使其进入 IPv4、IPv6 和 Source Routing 路径。
5. **结果验证与原有功能检查**：证明“一份输入、三份副本、三种封装、三条路径”，并确认原始 Demo 中已跑通的功能仍然正常。

1.4 配套材料

```
第2次实验材料_模态调度与异构封装/
├── 第二次实验指导_模态调度与异构封装.pdf
├── 代码/
│   ├── README.md
│   ├── 接口示例/
│   └── 调度开发骨架/
└── 运行结果/
    └── verified-scheduling.txt
```

路径	用途
`代码/调度开发骨架/polymorphic.p4`	P4 开发起点，含 TODO M1—M6
`代码/调度开发骨架/mycontroller.py`	P4Runtime 控制器开发起点
`代码/调度开发骨架/send-poly.py`	材料内的 UDP 发送脚本
`代码/调度开发骨架/receive-poly.py`	后续用于检查最终单一输出
`代码/接口示例/poly_headers.p4.inc`	PolyShim 和相关 header 的参考定义
`代码/接口示例/controller_multicast_example.py`	P4Runtime multicast 配置示例
`代码/接口示例/multicast_mapping_demo.py`	可直接运行的一入三出映射模型

`send-poly.py` 是 UDP 参考实现。选择 TCP 的小组需要相应调整收发脚本、Parser 分支和 CRC 字段范围，并在设计记录中说明报文格式。

第二部分：实验过程及要求

每组继续使用原团队 Git 仓库，可以使用 `代码/调度开发骨架/` 开始，也可以将其中新增的脚本和 TODO 合并到本组代码。具体的表名、action 拆分方式、multicast group 编号和代码组织可由小组自行设计；下文给出的编号和端口是与材料骨架配套的参考方案。

2.1 Parser 功能完善

Parser 需要为两类报文提供清晰的解析路径：

- 来自主机的原始实验报文；
- 携带 IPv4、IPv6 或 Source Routing 外层的多模态副本。

解析结果应能让后续 control 访问外层模态头、PolyShim、内层 IPv4、所选传输层头和定长实验数据。材料参考报文使用 16 字节数据，短消息补零，编码后超过 16 字节则拒绝发送。如果选择 UDP，可参考材料中目的端口 5000 的识别方式；如果选择 TCP，需要考虑 TCP `dataOffset` 和可选项对负载起始位置的影响。实验中可以由发送脚本构造无 TCP options 的固定格式报文，同时在报告中说明适用范围。

新增解析分支不应改变普通 ARP、ping、IPv6 和原 Source Routing 报文的处理结果。

2.2 入网报文识别与关联信息生成

源网关需要区分“需要调度的实验报文”和“按原有方式转发的普通报文”。可以组合使用入端口、IP 地址、传输层协议号和端口，也可以提出其他能清楚区分流量的方案。在材料拓扑中，h1→h2 时 S1 是源网关、S2 是目的网关；h2→h1 时角色互换。正向和反向报文应能得到正确的方向和网关角色。

序号和入口 CRC 需在报文复制前生成，这样三份副本才能共享同一组关联信息。CRC 可覆盖内层 IP 地址、协议号、传输层端口和定长数据等稳定字段；具体范围可结合小组方案确定，但入口和目的网关必须使用同一顺序、同一宽度重新计算。TTL、外层地址和逐跳变化的字段不适合放入跨路径比较范围。

2.3 报文复制与模态调度

材料的参考方案在 S1 和 S2 配置 multicast group 10，并使用下列副本映射：

```
(出口端口 2，副本编号 1)
(出口端口 3，副本编号 2)
(出口端口 4，副本编号 3)
```

Ingress 命中调度条件后设置 multicast group，PRE 再根据本机的 group 配置复制报文。S1 和 S2 的 PRE 状态相互独立，所以反向流量也需要对应配置。

小组可以继续使用 group 10，也可以使用其他 group 编号或等价的 P4Runtime 组织方式。验证时需要能从控制器配置和出口报文中看出“一份原始输入在源网关内部复制”，不能由主机分别构造三份模态报文代替。

2.4 异构封装与路径转发

副本进入 Egress 时，PRE 或流量管理模块已经确定 `standard_metadata.egress_port`。P4 程序可读取该字段选择封装，无需在 Egress 重新决定出口。与材料拓扑配套的参考映射为：

egress_port	modality_id	外层封装
2	1	IPv4，`protocol=253`
3	2	IPv6，`nextHdr=253`
4	3	Source Routing，`EtherType=0x1234`

如果小组调整了端口、模态编号或外层标识，需要同步调整拓扑、控制器、P4 处理和验证记录，让各部分可以互相印证。三份副本的 `sequence_number` 应保持一致，`modality_id` 用来区分它们所属的模态。

2.5 结果验证与原有功能检查

验证应回答以下问题：

1. 发送端是否只生成一份原始实验报文?
2. 三份副本是否由源网关复制，并携带相同序号?
3. 三份副本是否具有不同的模态编号和外层协议头?
4. 关键端口的 pcap 或交换机日志是否能说明三条真实路径?
5. 新增功能后，原始 Demo 的 IPv4、IPv6、ARP 和 Source Routing 功能是否仍然正常?

证据形式可以根据小组的调试方式选择，但应保留足以定位报文、模态和路径的命令、pcap、日志或截图，不建议只使用一张终端收包图作为全部证据。

2.6 代码 TODO 与任务对应

代码中的 `TODO M1` — `TODO M6` 是整个项目的实现索引。本次主要处理 M1—M4，M5、M6 留到下次实验。

TODO 与文件	参考内容	对应任务
`polymorphic.p4`： `TODO M1`	增加 PolyShim、定长数据和独立内层 IPv4，调整 Deparser 线序	Parser 功能完善；异构封装与路径转发
`polymorphic.p4`： `TODO M2`	为所选 UDP 或 TCP 实验报文增加 Parser 分支	Parser 功能完善
`polymorphic.p4`： Ingress `TODO M3`	识别源网关实验流量，生成序号和 CRC，并触发复制	入网报文识别与关联信息生成；报文复制与模态调度
`polymorphic.p4`： `TODO M4`	根据副本出口生成三种外层封装	异构封装与路径转发

TODO 与文件	参考内容	对应任务
`mycontroller.py`：控制面 TODO	下发调度表项和 multicast group，配置两个方向	报文复制与模态调度；结果验证与原有功能检查

`utils/p4runtime_lib/convert.py` 中的上游维护 TODO 不是本实验的填空内容。

2.7 过程记录

`class2/` 是小组在团队 Git 仓库中建立的阶段记录目录，不会由骨架或运行脚本自动生成。可参考以下结构：

```
class2/
├── packet-summary.txt
├── member-progress.txt
└── evidence/
```

- `packet-summary.txt` 用来记录发送数量、序号产生位置、模态编号、端口与外层协议的对应关系，以及对应证据文件名。
- `member-progress.txt` 记录本次各成员负责的代码、测试或分析内容，以及尚未解决的问题。
- `evidence/` 可以保存关键 pcap、日志、控制器输出和结果截图。程序生成的 `pcaps/` 和 `logs/` 仍位于实验项目目录，需要由小组选取与结论相关的文件。

建议在功能稳定后创建 `class2-multicast` tag，便于下次实验继续开发和排查问题。

2.8 作业提交与完成检查

本次作业用于了解小组的调度开发进度，不要求提交正式报告或完整代码。由组长提交 1—3 张能够辨认命令和结果的截图，内容应能说明一份输入已在源网关形成三份相关副本，并分别进入三种模态路径。

提交前可按下表检查：

- [ ] P4 和控制器通过语法或编译检查。
- [ ] 一份 UDP 或 TCP 实验输入在源网关形成三份副本。
- [ ] 三份副本的序号相同，模态编号不同。
- [ ] 三种外层协议与三条物理路径能够对应。
- [ ] 正向和反向的源网关都能执行报文复制。
- [ ] 原始 Demo 中的 IPv4、IPv6、ARP 和 Source Routing 功能仍然正常。
- [ ] 本次小组分工和实际进展已有记录。
- [ ] 作业截图已提交。

第三部分：实验指导

下列命令按材料内 UDP 参考方案编写。选择 TCP 的小组可保留整体步骤，将收发脚本、Parser 匹配条件和 CRC 字段替换为本组的 TCP 方案。

3.1 建立记录目录

在团队 Git 仓库根目录执行：

```
cd <团队Git仓库根目录>
mkdir -p class2/evidence
touch class2/packet-summary.txt class2/member-progress.txt
```

`touch` 只创建空文件，后续需要根据本组实际设计和运行结果填写。

3.2 运行端口映射模型

```
cd <本课目录>/代码/接口示例
python3 multicast_mapping_demo.py --sequence 1 --payload class2-test
python3 -m py_compile controller_multicast_example.py multicast_mapping_demo.py
```

该模型用来观察一份输入与三个端口、三个模态的映射关系，它不代表 P4 功能已经完成。参考输出如下：

```
INPUT count=1 sequence=1 protected_hex=636c617373322d746573740000000000
REPLICA egress_port=2 modality_id=1 encapsulation=IPv4 sequence=1 path=s1-s11-s12-s2
REPLICA egress_port=3 modality_id=2 encapsulation=IPv6 sequence=1 path=s1-s21-s22-s2
REPLICA egress_port=4 modality_id=3 encapsulation=SourceRouting sequence=1 path=s1-s31-s32-s2
SUMMARY input=1 replicas=3 same_sequence=yes
```

3.3 复制开发骨架并检查起点

先将 `代码/调度开发骨架/` 复制到团队实验项目，或将本次需要的文件合并到现有项目。不要直接改动 `class1-baseline` tag 指向的版本。

```
cd <实验项目目录>
make build
python3 -m py_compile mycontroller.py send-poly.py receive-poly.py
```

未完成 TODO 的骨架应当可以编译，并能继续运行原始三条独立路径。如果当前终端提示找不到 `p4c`、`simple_switch_grpc` 等命令，再在该终端执行一次：

```
source ~/p4setup.bash
```

3.4 完成 M1：协议头与 Deparser

1. 对照 `代码/接口示例/poly_headers.p4.inc` 理解各字段。
2. 在 `polymorphic.p4` 中加入 PolyShim、定长数据和独立内层 IPv4 header。
3. 把新 header 加入 headers struct，再按实际线序调整 Deparser。
4. 执行 `make build`，先处理 header 类型、位宽和发出顺序问题。

参考线序：

```
Ethernet / 外层模态头 / PolyShim / 内层 IPv4 /
UDP 或 TCP / protected_data
```

3.5 完成 M2：Parser

先画出本组实际报文的解析路径，再修改 Parser。UDP 参考方案可以按 IPv4 `protocol=17` 和目的端口 5000 进入定长数据分支；TCP 方案应使用 `protocol=6` 和本组选定的端口，并处理或限定 TCP header 长度。

三种已封装报文的 Parser 路径最终都应进入 PolyShim、内层 IPv4、传输层和定长数据。完成后执行：

```
make build
```

然后先运行普通 ping、IPv6 和 Source Routing，确认新 Parser 分支没有拦截原有报文。

3.6 完成 M3：Ingress 识别、序号、CRC 和复制

可按以下顺序实现并逐步编译：

```
识别源网关和实验流量
-> 确定方向
-> 保存内层 IPv4 和传输层必要信息
-> 分配一次 sequence_number
-> 按本组确定的字段顺序计算 source_crc32
-> 设置 multicast group
```

不要把序号递增放在 Egress，因为 Egress 会对三份副本分别执行。参考骨架使用 group 10：

```
standard_metadata.mcast_grp = 10;
```

每完成一小步执行 `make build`，并对照 P4Info 确认需要下发的表和 action 名称。

3.7 完成 M4：P4Runtime multicast 与 Egress 封装

参考 `controller_multicast_example.py` 向 `mycontroller.py` 加入 multicast group 配置。与材料拓扑配套的配置为：

```
group 10
port 2, instance 1
port 3, instance 2
port 4, instance 3
```

分别为 S1 和 S2 安装该配置，再安装正向和反向调度表项。

Egress 中读取 `standard_metadata.egress_port`，按端口生成外层 IPv4、IPv6 或 Source Routing 头，并写入对应 `modality_id`。完成后再次执行：

```
make build
python3 -m py_compile mycontroller.py
```

3.8 启动修改后的项目

终端 A：

```
cd <实验项目目录>
make
```

终端 B：

```
cd <实验项目目录>
python3 mycontroller.py
```

若控制器报表名、action 名或参数不存在，先对照本次编译生成的 P4Info，不要直接沿用其他版本代码中的数字 ID。

3.9 发送一份实验报文

材料内 UDP 脚本的正向命令：

```
mininet> h1 python3 send-poly.py 10.0.2.2 class2-test
```

反向命令：

```
mininet> h2 python3 send-poly.py 10.0.1.1 reverse-test
```

发送端每次应只打印一条 `POLY_SEND`。如果本组选择 TCP，在此处使用本组收发脚本，并保留“主机每次只发一份”的证据。

3.10 检查 pcap 和交换机日志

在另一个终端执行：

```
cd <实验项目目录>
ls pcaps | sort
tcpdump -nn -e -xx -r pcaps/<s1端口2出口文件>.pcap
tcpdump -nn -e -xx -r pcaps/<s1端口3出口文件>.pcap
tcpdump -nn -e -xx -r pcaps/<s1端口4出口文件>.pcap
```

材料参考映射的判断依据：

```
端口2: IPv4, protocol 253
端口3: IPv6, next header 253
端口4: EtherType 0x1234
```

继续检查 S2 端口 2、3、4 的入口 pcap，核对副本数量、序号、模态编号和外层协议。如果报文没有到达预期端口，可使用以下方式跟踪某台交换机：

```
tail -f logs/s1.log
```

将用于得出结论的 pcap、日志或截图复制到 `class2/evidence/`，并在 `packet-summary.txt` 中写明文件名和对应结论。

3.11 检查原有功能

材料 UDP 参考方案只调度选定的 IPv4/UDP 实验流量。完成新功能后，重新运行：

```
mininet> pingall
mininet> h1 ping6 -c 2 fe80::5678%eth0
```

再按原始 Demo 的命令检查正向和反向 Source Routing。选择 TCP 的小组也需要检查非实验 TCP 或其他普通流量没有被误调度。

如果 IPv4、IPv6、ARP 或 Source Routing 失败，先检查 Parser 匹配条件、调度表项和 Egress 封装分支，再继续后续实验。

3.12 整理结果并保存版本

在 `class2/packet-summary.txt` 中整理以下内容：

1. 主机只发送一份的证据；
2. 序号和 CRC 的产生位置；
3. 三份副本的序号和模态编号；
4. 三个出口、外层协议和路径的对应关系；
5. 关键证据文件的位置。

确认当前 commit 能够重新编译和运行后，创建 tag：

```
git tag class2-multicast
```

若 tag 已存在，先用 `git show class2-multicast` 检查它是否指向本组确认过的 commit，不要直接覆盖。

3.13 清理运行环境

```
mininet> exit
```

```
make stop
make clean
```

3.14 参考运行现象

材料内参考实现使用 UDP 发送一份 `smoke-two` 后，S1 三个出口可观察到：

```
s1-eth2_out: IPv4 10.0.1.1 > 10.0.2.2, protocol 253
s1-eth3_out: IPv6 fe80::1234 > fe80::5678, next header 253
s1-eth4_out: EtherType 0x1234, Source Routing
```

三份副本的 `sequence_number` 相同，`modality_id` 分别为 1、2、3。初始骨架编译时可能出现原始 Demo 中未使用 ARP 常量的 warning，但不应有 error。

本次不把目的主机只收到一份作为通过条件。如果当前只完成调度，目的网关可能丢弃内部副本，或尚未完成透明还原。这部分在下次实验继续实现。

更完整的纯文本参考结果见 `运行结果/verified-scheduling.txt`。